

1. Introduction to Linear Algebra for Machine Learning

Linear Algebra is one of the **most important mathematical foundations** of Machine Learning. Almost every ML algorithm—from Linear Regression to Neural Networks—uses vectors, matrices, and matrix operations.

a) Importance of Linear Algebra in Machine Learning

Machine Learning works with **data**. Data is usually represented as **vectors** and **matrices**.

For example:

Student	Maths	Physics	Chemistry
A	80	75	90
B	70	88	76
C	92	81	95

This dataset can be written as a matrix:

```
[  
  \begin{bmatrix}  
    80 & 75 & 90 \\  
    70 & 88 & 76 \\  
    92 & 81 & 95 \\  
  \end{bmatrix}  
]
```

Almost every ML library (NumPy, Pandas, TensorFlow, PyTorch) internally performs matrix operations.

Why is Linear Algebra important?

1. Data Representation

Datasets are stored as matrices.

Example:

Height	Weight	Age
170	65	21
180	72	23
165	58	20

Shape

(3 × 3)

Three rows (samples)

Three columns (features)

2. Images are Matrices

A grayscale image is just numbers.

Example

```
[  
  [255, 200, 150],  
  [120, 180, 255],  
  [ 50,  80, 100]  
]
```

Each number represents pixel brightness.

Color images use three matrices:

- Red
- Green
- Blue

3. Machine Learning Models

Suppose

House Price =
Area $\times$ Weight1 +
Bedrooms $\times$ Weight2 +
Age $\times$ Weight3

Instead of writing this manually,

Linear Algebra represents it as

Prediction = XW

where

X = Features

W = Weights

This makes calculations extremely fast.

4. Deep Learning

Neural Networks repeatedly compute

Output = $WX + b$

where

W = Weight Matrix

X = Input Vector

b = Bias

Millions of these calculations happen every second.

5. Computer Vision

Tasks like

- Face Detection
- Image Recognition
- Object Detection

all use matrices.

6. Natural Language Processing (NLP)

Words become vectors.

Example

King

↓

[0.82, 1.56, -0.32, ...]

These are called **word embeddings**.

7. Recommendation Systems

Netflix

YouTube

Amazon

Spotify

all use matrices to determine what you may like.

Summary

Linear Algebra helps us

- Represent data
- Store images
- Train ML models
- Build Neural Networks
- Process text
- Analyze large datasets

Without Linear Algebra, modern AI would not exist.

b) Scalars, Vectors, and Matrices

These are the building blocks of Linear Algebra.

1. Scalar

A scalar is a **single number**.

Examples

5

-3

0

2.75

In ML

Learning Rate = 0.01

Age = 21

Temperature = 36.5

Python

```
x = 10
```

```
print(type(x))
```

Output

```
<class 'int'>
```

NumPy

```
import numpy as np
```

```
x = np.array(5)
```

```
print(x)
```

```
print(x.shape)
```

Output

```
5
```

```
()
```

Notice the shape is empty because it is a scalar.

2. Vector

A vector is a **1-dimensional array of numbers**.

Example

```
[2,4,6]
```

Think of it as one row of data.

Example

Height
Weight
Age

↓

[170, 65, 21]

NumPy

```
import numpy as np
```

```
v = np.array([170,65,21])
```

```
print(v)
```

```
print(v.shape)
```

Output

[170 65 21]

(3,)

This means

3 elements

Vector Visualization

Origin

•

\
\
\
----->

Vector

Vectors have

- Magnitude (length)
 - Direction
-

3. Matrix

A matrix is a **2-dimensional array**.

Example

```
[  
  [1,2],  
  [3,4]  
]
```

NumPy

```
A = np.array(  
  [1,2],  
  [3,4]  
)
```

```
print(A)  
print(A.shape)
```

Output

```
[[1 2]  
 [3 4]]
```

```
(2,2)
```

Meaning

2 Rows

2 Columns

Example Dataset

Height Weight Age

170 65 21

180 72 23

165 58 20

becomes

```
dataset = np.array([
    [170,65,21],
    [180,72,23],
    [165,58,20]
])
```

Shape

(3,3)

Difference

Object	Dimension	Example
Scalar	0D	5
Vector	1D	[2, 4, 6]
Matrix	2D	[[1, 2], [3, 4]]

c) Basic Vector Operations

We'll use the vectors:

```
import numpy as np
```

```
A = np.array([2,4,6])
```

```
B = np.array([1,3,5])
```

1. Vector Addition

Add corresponding elements.

```
A = [2,4,6]
```

```
B = [1,3,5]
```

↓

```
[3,7,11]
```

Python

```
A + B
```

Output

```
array([3,7,11])
```

2. Vector Subtraction

Subtract corresponding elements.

```
[2,4,6]
```

-

```
[1,3,5]
```

=

[1,1,1]

Python

A - B

Output

array([1,1,1])

3. Scalar Multiplication

Multiply every element by the scalar.

Example

$3 \times [2,4,6]$

↓

[6,12,18]

Python

$3 * A$

Output

array([6,12,18])

More Examples

`A = np.array([10,20,30])`

`print(A + 5)`

Output

```
[15 25 35]
```

This is called **broadcasting** in NumPy.

```
print(A * 2)
```

Output

```
[20 40 60]
```

```
print(A / 10)
```

Output

```
[1. 2. 3.]
```

Complete Example

```
import numpy as np
```

```
A = np.array([2,4,6])
```

```
B = np.array([1,3,5])
```

```
print("A =", A)
```

```
print("B =", B)
```

```
print("Addition:", A + B)
```

```
print("Subtraction:", A - B)
```

```
print("Scalar Multiplication:", 2 * A)
```

Output

```
A = [2 4 6]
```

```
B = [1 3 5]
```

```
Addition: [ 3  7 11]
```

Subtraction: [1 1 1]

Scalar Multiplication: [4 8 12]

Real ML Example

Suppose you have a student's features:

```
student = np.array([170, 65, 21])
```

Where:

- 170 = Height (cm)
- 65 = Weight (kg)
- 21 = Age (years)

If all measurements are accidentally recorded in double scale:

```
student = 2 * student
```

Output:

```
[340 130 42]
```

If you compare two students:

```
student1 = np.array([170,65,21])
```

```
student2 = np.array([165,60,22])
```

```
difference = student1 - student2
```

```
print(difference)
```

Output:

```
[ 5  5 -1]
```

This tells you:

- Student 1 is **5 cm taller**
 - Student 1 is **5 kg heavier**
 - Student 1 is **1 year younger**
-

Key Takeaways

- **Linear Algebra** is the mathematical language of Machine Learning.
 - A **scalar** is a single value (0D).
 - A **vector** is a one-dimensional collection of values (1D).
 - A **matrix** is a two-dimensional table of values (2D).
 - The most common vector operations are:
 - **Addition**: Add corresponding elements.
 - **Subtraction**: Subtract corresponding elements.
 - **Scalar multiplication**: Multiply every element by the same number.
 - In Machine Learning, datasets, images, model parameters, and predictions are all represented and manipulated using vectors and matrices.
-

2. Vector Spaces and Subspaces

What is a Vector Space?

A **vector space** is a collection of vectors that follows certain mathematical rules.

The vectors in a vector space can be:

- Added together
- Multiplied by scalars
- Still remain inside the same vector space

Simply put,

A vector space is a set of vectors where vector addition and scalar multiplication are always valid.

Example 1

The following vectors belong to the vector space $\mathbb{R}^2$ (2-dimensional real space):

[1, 2]

[3, 5]

[-2, 7]

[0, 0]

Adding two vectors:

[1,2] + [3,5]

=

[4,7]

The result is still in $\mathbb{R}^2$.

Scalar multiplication:

$3 \times [2,4]$

=

[6,12]

Still inside $\mathbb{R}^2$.

Common Vector Spaces

Vector Space	Description
$\mathbb{R}^1$	One-dimensional vectors
$\mathbb{R}^2$	Two-dimensional vectors

$\mathbb{R}^3$	Three-dimensional vectors
$\mathbb{R}^n$	n-dimensional vectors

Examples

$$\mathbb{R}^1 \rightarrow [5]$$

$$\mathbb{R}^2 \rightarrow [2,4]$$

$$\mathbb{R}^3 \rightarrow [2,4,6]$$

$$\mathbb{R}^5 \rightarrow [1,3,5,7,9]$$

Why Vector Spaces Matter in ML

Every data point is a vector.

Example

Height	Weight	Age
170	65	21

becomes

$$[170,65,21]$$

This vector lives in $\mathbb{R}^3$.

If a dataset has 100 features,

each sample belongs to

$$\mathbb{R}^{100}$$

What is a Subspace?

A **subspace** is a smaller vector space inside another vector space.

Example

Suppose

$$\mathbb{R}^2$$

contains every possible 2D vector.

A straight line through the origin is a subspace.

Example

$$y = 2x$$

Vectors on this line:

$$[1, 2]$$

$$[2, 4]$$

$$[-3, -6]$$

These vectors still satisfy all vector space rules.

Example of NOT a Subspace

$$y = 2x + 1$$

This line does **not** pass through the origin.

Therefore,

it is **not** a subspace.

Conditions for a Subspace

A subset is a subspace if:

- Contains the zero vector
 - Closed under vector addition
 - Closed under scalar multiplication
-

ML Example

Suppose your dataset has

Height
Weight
Age
Salary

Each person is represented as

[170,65,21,50000]

Every sample belongs to

$\mathbb{R}^4$

Feature engineering often creates lower-dimensional subspaces.

b) Types of Matrices

Matrices come in different forms depending on their structure.

1. Row Matrix

One row

[2 4 6]

Shape

1×3

NumPy

```
A = np.array([[2,4,6]])
```

2. Column Matrix

One column

```
[  
2  
4  
6  
]
```

Shape

3×1

NumPy

```
A = np.array([[2],  
              [4],  
              [6]])
```

3. Square Matrix

Rows = Columns

Example

```
[  
1 2  
3 4  
]
```

Shape

2×2

4. Rectangular Matrix

Rows $\neq$ Columns

```
[  
1 2 3  
4 5 6  
]
```

Shape

2×3

5. Zero Matrix

Every element is zero.

```
[  
0 0  
0 0  
]
```

NumPy

```
np.zeros((2,2))
```

Output

```
[[0. 0.]  
 [0. 0.]]
```

6. Diagonal Matrix

Only diagonal elements are non-zero.

```
[
3 0 0
0 5 0
0 0 7
]
```

NumPy

```
np.diag([3,5,7])
```

7. Identity Matrix

Diagonal values are 1.

```
[
1 0 0
0 1 0
0 0 1
]
```

NumPy

```
np.eye(3)
```

Output

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Why is the Identity Matrix Important?

It behaves like the number **1** in multiplication.

Example

$$A \times I = A$$

Example

$$A =$$

$$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$$

$$I =$$

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$A \times I$$

$$=$$

$$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$$

8. Upper Triangular Matrix

Everything below the diagonal is zero.

$$\begin{bmatrix} 2 & 5 & 8 \\ 0 & 3 & 4 \\ 0 & 0 & 7 \end{bmatrix}$$

9. Lower Triangular Matrix

Everything above the diagonal is zero.

```
[  
2 0 0  
5 3 0  
8 4 7  
]
```

Summary

Matrix Type	Example
Row	$1 \times n$
Column	$n \times 1$
Square	$n \times n$
Rectangular	$m \times n$
Zero	All zeros
Diagonal	Only diagonal values
Identity	Diagonal = 1
Upper Triangular	Below diagonal = 0
Lower Triangular	Above diagonal = 0

c) Matrix Operations

We'll use

```
import numpy as np
```

```
A = np.array([[1,2],  
              [3,4]])
```

```
B = np.array([[5,6],  
              [7,8]])
```

1. Matrix Addition

Add corresponding elements.

```
[  
1 2  
3 4  
]
```

+

```
[  
5 6  
7 8  
]
```

=

```
[  
6 8  
10 12  
]
```

Python

A + B

2. Matrix Subtraction

Subtract corresponding elements.

```
[  
5 6  
7 8  
]
```

-

```
[
```

1 2
3 4
]

=

[
4 4
4 4
]

Python

B - A

3. Matrix Multiplication

Unlike addition, multiplication follows the **row × column rule**.

genui{"linear_algebra_optimization_learning_block":{"type_id":"MATRIX_MULTIPLICATION_ROW_COLUMN_RULE"}}

Rule

If

$A = (m \times n)$

$B = (n \times p)$

Then

$AB = (m \times p)$

The number of **columns of A** must equal the **rows of B**.

Example

```
A =  
[  
 1 2  
 3 4  
]
```

```
B =  
[  
 5 6  
 7 8  
]
```

First element

$(1 \times 5) + (2 \times 7)$

=

19

Complete result

```
[  
 19 22  
 43 50  
]
```

NumPy

`A @ B`

or

`np.matmul(A,B)`

Output

```
[[19 22]  
 [43 50]]
```

Difference Between * and @

$A * B$

Output

```
[[ 5 12]
 [21 32]]
```

This is **element-wise multiplication**.

$A @ B$

Output

```
[[19 22]
 [43 50]]
```

This is **true matrix multiplication**.

Matrix Multiplication is NOT Commutative

Usually,

$AB \neq BA$

This is one of the most important concepts in Linear Algebra.

d) Transpose of a Matrix

The transpose of a matrix converts **rows into columns** and **columns into rows**.

Notation

A^T

Example

Original

```
[  
1 2 3  
4 5 6  
]
```

Transpose

```
[  
1 4  
2 5  
3 6  
]
```

Rows become columns.

NumPy

```
A = np.array([[1,2,3],  
              [4,5,6]])
```

```
print(A.T)
```

Output

```
[[1 4]  
 [2 5]  
 [3 6]]
```

Shape After Transpose

If

$$A = (2 \times 3)$$

then

$$A^T = (3 \times 2)$$

Example

Original

2 Rows

3 Columns

↓

Transpose

3 Rows

2 Columns

Why is Transpose Important in Machine Learning?

Transpose is used in:

- Matrix multiplication
- Linear Regression
- Principal Component Analysis (PCA)
- Covariance matrices
- Neural Networks
- Feature transformations

Example:

A dataset with shape

$$100 \times 5$$

means:

- 100 samples
- 5 features

After transpose:

5×100

Now each row represents a feature instead of a sample.

Complete NumPy Example

```
import numpy as np
```

```
A = np.array([[1,2],  
              [3,4]])
```

```
B = np.array([[5,6],  
              [7,8]])
```

```
print("Addition:")  
print(A + B)
```

```
print("\nSubtraction:")  
print(B - A)
```

```
print("\nElement-wise Multiplication:")  
print(A * B)
```

```
print("\nMatrix Multiplication:")  
print(A @ B)
```

```
print("\nTranspose:")  
print(A.T)
```

Output

```
Addition:  
[[ 6  8]  
 [10 12]]
```

Subtraction:

```
[[4 4]  
[4 4]]
```

Element-wise Multiplication:

```
[[ 5 12]  
[21 32]]
```

Matrix Multiplication:

```
[[19 22]  
[43 50]]
```

Transpose:

```
[[1 3]  
[2 4]]
```

Absolutely. Since you're learning **Linear Algebra for Machine Learning**, here's the order I recommend. Each topic builds on the previous one.

Matrix Operations Roadmap

1. Matrix Addition

Condition:

- Both matrices must have the same dimensions.

Example:

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} + \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 10 & 12 \end{bmatrix}$$

Python:

- for i in range(n):
- for j in range(n):

- $C[i][j] = A[i][j] + B[i][j]$
-

2. Matrix Subtraction

Same rule.

$$A - B$$

Python

- for i in range(n):
 - for j in range(n):
 - $C[i][j] = A[i][j] - B[i][j]$
-

3. Scalar Multiplication

Multiply every element by a number.

Example

$$3 \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 9 & 12 \end{bmatrix}$$

Python

- scalar = 3
-

- for i in range(n):
 - for j in range(n):
 - A[i][j] *= scalar
-

4. Matrix Multiplication

Already learned.

Uses **three loops**.

Time Complexity

$$O(n^3)$$

5. Transpose

Very important in ML.

Transpose means

Rows become Columns

Example

Original

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Transpose

$$\begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

Python

 Python

 Run

```
transpose = []

for j in range(n):
    row = []
    for i in range(n):
        row.append(A[i][j])
    transpose.append(row)

print(transpose)
```

or

 Python

 Run

```
for i in range(n):
    for j in range(i+1, n):
        A[i][j], A[j][i] = A[j][i], A[i][j]
```



This second method works **only for square matrices** and does it **in place**.

6. Identity Matrix

An identity matrix has

- 1 on the diagonal
- 0 elsewhere

Example

Example

Upgrade

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Python

Python



Run

```
I = []

for i in range(n):
    row = []
    for j in range(n):
        if i == j:
            row.append(1)
        else:
            row.append(0)
    I.append(row)
```

9. Determinant

Very important.

9. Determinant ★★★★★

Very important.

For

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$\det(A) = ad - bc$$

Example

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$1(4) - 2(3) = -2$$

A matrix is invertible only if

$$\det(A) \neq 0$$

10. Inverse

One of the most important concepts.

Definition

One of the most important concepts.

Definition

$$AA^{-1} = I$$

Example

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Inverse

$$A^{-1} = \frac{1}{-2} \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix}$$

We'll derive this from scratch later.

We'll derive this from scratch later.

14. Eigenvalues & Eigenvectors

The heart of Linear Algebra.

Used in

- PCA
- SVD
- PageRank

- Image Compression
- Machine Learning

$$Av = \lambda v$$

Definition

Given a square matrix (A), an **eigenvector** is a non-zero vector ($\mathbf{v}$) that changes only in magnitude (not direction) when multiplied by (A).

Mathematically,

$$A\mathbf{v} = \lambda\mathbf{v}$$

where:

- A = square matrix
- $\mathbf{v}$ = eigenvector
- λ = eigenvalue

Intuition

Normally, matrix multiplication changes both the **length** and **direction** of a vector.

Example:

Before

↑
/
/

After multiplying



An eigenvector is special because its **direction remains the same**.

Before



After



Only the length changes.

Properties

- Defined only for **square matrices** ($(n \times n)$).
 - A matrix can have multiple eigenvalues.
 - Eigenvectors corresponding to different eigenvalues are linearly independent (under suitable conditions).
 - If $(\lambda = 0)$, the matrix is singular (not invertible).
 - The determinant equals the product of all eigenvalues.
 - The trace equals the sum of all eigenvalues.
-

2. Computation of Eigenvalues and Eigenvectors

Suppose

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Step 1

Start from

$$A\mathbf{v} = \lambda\mathbf{v}$$

Move everything to one side:

$$(A - \lambda I)\mathbf{v} = 0$$

where I is the identity matrix.

Step 2

Construct

$$A - \lambda I = \begin{bmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{bmatrix}$$

Step 3

Take the determinant:

$$\begin{vmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{vmatrix} = 0$$

Expand:

$$(2 - \lambda)^2 - 1 = 0$$

Simplify:

$$\lambda^2 - 4\lambda + 3 = 0$$

Solve:

$$(\lambda - 3)(\lambda - 1) = 0$$

Eigenvalues:

$$\lambda_1 = 3, \quad \lambda_2 = 1$$

Step 4

Find eigenvectors.

For

$$\lambda = 3$$

Substitute back into

$$(A - \lambda I)\mathbf{v} = \mathbf{0}$$

Solve the resulting system to obtain

$$\mathbf{v} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

For

$$\lambda = 1$$

you obtain

$$\mathbf{v} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

3. Diagonalization of Matrices

A matrix is **diagonalizable** if it can be written as

$$A = PDP^{-1}$$

where:

- P = matrix whose columns are eigenvectors
- D = diagonal matrix containing eigenvalues
- P^{-1} = inverse of P

Example:

$$D = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

Why diagonalize?

Diagonal matrices are much easier to work with.

For example,

Instead of calculating

$$A^{100}$$

you can compute

$$A^{100} = P D^{100} P^{-1}$$

Since

$$D^{100} = \begin{bmatrix} 3^{100} & 0 \\ 0 & 1^{100} \end{bmatrix}$$

the computation becomes much simpler.

4. Applications in Machine Learning: Principal Component Analysis (PCA)

PCA is one of the most important applications of eigenvalues and eigenvectors.

Problem

Imagine a dataset with:

- Height
- Weight
- Age
- Salary
- Experience

Some of these features are highly correlated.

Example:

Height ↑

Weight ↑

Both increase together.

This creates redundancy.

Goal of PCA

Reduce the number of features while preserving as much information (variance) as possible.

Example:

Original:

Height

Weight

Age

Salary

Experience

↓

PCA

↓

PC1

PC2

The dataset goes from **5 dimensions to 2 dimensions**.

PCA Steps

1. Standardize the data.
 2. Compute the covariance matrix.
 3. Compute eigenvalues and eigenvectors of the covariance matrix.
 4. Sort eigenvalues in descending order.
 5. Select the top (k) eigenvectors.
 6. Project the data onto these eigenvectors.
-

Why Eigenvectors?

The eigenvectors define the **new axes (principal components)**.

The eigenvalues tell us **how much variance** each principal component explains.

Example:

Eigenvalue	Explained Variance
12.4	78%
2.1	13%
0.8	5%
0.3	2%

The first principal component captures the largest amount of information.

Applications of PCA

- Dimensionality reduction
 - Data visualization (2D or 3D)
 - Noise reduction
 - Image compression
 - Feature extraction
 - Faster machine learning training
 - Removing multicollinearity
-

Summary

Topic	Key Idea	ML Importance
Definition	$(A\mathbf{v}=\lambda\mathbf{v})$	★★★★ ★★
Eigenvalues	Amount of scaling along an eigenvector	★★★★ ★★

Eigenvectors

Directions that remain unchanged by the transformation

**Diagonalization**

Express (A) as (PDP^{-1}) for easier computations

**PCA**

Uses eigenvectors of the covariance matrix to reduce dimensions

**Suggested learning order**

To fully understand these topics, I recommend studying them in this sequence:

1. Matrix multiplication
2. Determinant
3. Inverse
4. Identity matrix
5. Characteristic equation
6. Eigenvalues
7. Eigenvectors
8. Diagonalization
9. Covariance matrix
10. PCA (implemented from scratch, then with NumPy)

Introduction to Calculus for Machine Learning

a) Importance of Calculus in Machine Learning

Calculus is the mathematics of change. In Machine Learning, models learn by adjusting their parameters to minimize prediction error. Calculus provides the tools required for optimization and learning.

Importance

Optimization: ML algorithms minimize a loss (cost) function.

$$\text{Loss} = (y - \hat{y})^2$$

Gradient Descent: Parameters are updated using derivatives.

$$\theta = \theta - \alpha(\partial J/\partial \theta)$$

Where:

- θ = model parameter
- α = learning rate
- J = cost function

Neural Networks: Backpropagation uses the chain rule to compute gradients.

Probability and Statistics: Continuous probability distributions involve integration.

Applications in ML

- Linear Regression
 - Logistic Regression
 - Neural Networks
 - Gradient Descent
 - Support Vector Machines
 - Bayesian Machine Learning
 - Reinforcement Learning (advanced)
-

b) Basic Concepts: Limits and Continuity

Limits

A limit describes the value that a function approaches as the input approaches a particular point.

General notation:

$$\lim_{x \rightarrow a} f(x)$$

Example:

$$\lim_{x \rightarrow 2} (x + 3) = 5$$

This means that as x gets closer to 2, the value of $x + 3$ gets closer to 5.

Continuity

A function $f(x)$ is continuous at $x = a$ if:

1. $f(a)$ exists

2. $\lim_{x \rightarrow a} f(x)$ exists
3. $\lim_{x \rightarrow a} f(x) = f(a)$

Example of a continuous function:

$$f(x) = x^2$$

Continuous functions have no breaks, holes, or jumps.

c) Introduction to Derivatives

A derivative measures the instantaneous rate of change of a function.

Definition

$$f'(x) = \lim_{h \rightarrow 0} [f(x + h) - f(x)] / h$$

Interpretation

- Geometrically: slope of the tangent line
- Physically: instantaneous velocity
- In ML: sensitivity of the loss function to parameter changes

Example

$$f(x) = x^2$$

$$f'(x) = 2x$$

At $x = 3$:

$$f'(3) = 6$$

This means the slope of the curve at $x = 3$ is 6.

Rules of Differentiation

a) Basic Rules

Constant Rule

$$d/dx(c) = 0$$

Example:

$$d/dx(8) = 0$$

Power Rule

$$d/dx(x^n) = n \cdot x^{n-1}$$

Examples:

$$d/dx(x^5) = 5x^4$$

$$d/dx(x^2) = 2x$$

Constant Multiple Rule

$$d/dx(c \cdot f(x)) = c \cdot f'(x)$$

Example:

$$d/dx(7x^3) = 21x^2$$

Sum Rule

$$d/dx(f + g) = f' + g'$$

Difference Rule

$$d/dx(f - g) = f' - g'$$

b) Product Rule, Quotient Rule, and Chain Rule

Product Rule

$$d/dx[f(x)g(x)] = f'(x)g(x) + f(x)g'(x)$$

Example:

$$f(x) = x^2(x + 1)$$

Let:

- $u = x^2$
- $v = x + 1$

Then:

$$f'(x) = 2x(x + 1) + x^2(1)$$

$$= 2x^2 + 2x + x^2$$

$$= 3x^2 + 2x$$

Quotient Rule

$$d/dx[f(x)/g(x)] = [f'(x)g(x) - f(x)g'(x)] / [g(x)]^2$$

Example:

$$y = x / (x + 1)$$

$$dy/dx = [(1)(x + 1) - x(1)] / (x + 1)^2$$

$$= 1 / (x + 1)^2$$

Chain Rule

If $y = f(g(x))$, then:

$$dy/dx = (dy/dg) \times (dg/dx)$$

Example:

$$y = (3x + 2)^5$$

Outer derivative:

$$5(3x + 2)^4$$

Inner derivative:

$$3$$

Therefore:

$$dy/dx = 5(3x + 2)^4 \times 3$$

$$= 15(3x + 2)^4$$

The chain rule is the foundation of backpropagation in neural networks.

c) Higher-Order Derivatives

The second derivative is written as:

$$f''(x)$$

The third derivative is:

$$f'''(x)$$

Example

$$f(x) = x^3$$

First derivative:

$$f'(x) = 3x^2$$

Second derivative:

$$f''(x) = 6x$$

Third derivative:

$$f'''(x) = 6$$

Interpretation

- $f'(x)$: rate of change
- $f''(x)$: curvature or acceleration
- $f'''(x)$: rate of change of acceleration

In optimization, the second derivative helps determine minima and maxima.

d) Basics of Integration and Its Rules

Integration is the reverse process of differentiation.

Indefinite Integral

$$\int f(x) dx$$

Example:

$$\int 3x^2 dx = x^3 + C$$

where C is the constant of integration.

Basic Integration Rules

Power Rule

$$\int x^n dx = x^{n+1} / (n + 1) + C, \text{ for } n \neq -1$$

Example:

$$\int x^2 dx = x^3/3 + C$$

Constant Rule

$$\int c dx = cx + C$$

Example:

$$\int 5 dx = 5x + C$$

Constant Multiple Rule

$$\int c \cdot f(x) dx = c \int f(x) dx$$

Example:

$$\int 4x^2 dx = 4(x^3/3) + C$$

$$= 4x^3/3 + C$$

Sum Rule

$$\int [f(x) + g(x)] dx = \int f(x) dx + \int g(x) dx$$

Difference Rule

$$\int [f(x) - g(x)] dx = \int f(x) dx - \int g(x) dx$$

Definite Integral

A definite integral has limits and gives a numerical value.

$\int_a^b f(x) \, dx$

Example:

$\int_0^2 x^2 \, dx$

$= [x^3/3]_0^2$

$= 8/3$

Geometric Meaning

A definite integral represents the signed area under the curve between $x = a$ and $x = b$.

Summary

Topic	Key Formula	ML Importance
Limits	$\lim_{x \rightarrow a} f(x)$	Foundation
Continuity	$\lim_{x \rightarrow a} f(x) = f(a)$	Smooth functions
Derivative	$f'(x)$	Optimization
Power Rule	$d/dx(x^n) = n \cdot x^{n-1}$	Basic differentiation
Product Rule	$(fg)' = f'g + fg'$	Composite models
Quotient Rule	$(f/g)' = (f'g - fg')/g^2$	Rational functions
Chain Rule	$dy/dx = (dy/dg)(dg/dx)$	Backpropagation

Higher Derivatives $f''(x), f'''(x)$

Curvature analysis

Integration $\int f(x) dx$

Probability and area

Definite Integral $\int_a^b f(x) dx$

Continuous distributions

Partial Derivatives, Gradient, and Gradient Descent (Machine Learning Notes)

Partial Derivatives

Definition

A **partial derivative** is the derivative of a function with respect to **one variable**, while keeping all the other variables constant.

Unlike ordinary derivatives, which deal with functions of one variable, partial derivatives are used for functions with **two or more variables**.

Notation

$\partial f / \partial x \rightarrow$ Partial derivative of **f** with respect to **x**

$\partial f / \partial y \rightarrow$ Partial derivative of **f** with respect to **y**

The symbol ∂ (called "partial") is used instead of **d** because the function has multiple variables.

Example

Suppose,

$$f(x, y) = x^2 + 3xy + y^2$$

Partial Derivative with Respect to x

Treat **y** as a constant.

$$\partial/\partial x(x^2) = 2x$$

$$\partial/\partial x(3xy) = 3y$$

$$\partial/\partial x(y^2) = 0$$

Therefore,

$$\partial f/\partial x = 2x + 3y$$

Partial Derivative with Respect to y

Treat **x** as a constant.

$$\partial/\partial y(x^2) = 0$$

$$\partial/\partial y(3xy) = 3x$$

$$\partial/\partial y(y^2) = 2y$$

Therefore,

$$\partial f/\partial y = 3x + 2y$$

Another Example

Given,

$$f(x, y) = 4x^2 + 2xy + 5y^2$$

Find the partial derivatives.

With respect to x,

$$\partial f/\partial x = 8x + 2y$$

With respect to y,

$$\partial f / \partial y = 2x + 10y$$

Why are Partial Derivatives Important?

Machine learning models usually have many parameters.

Example:

$$\text{Cost} = f(w_1, w_2, w_3, \dots, w_n)$$

To reduce the cost, we calculate the partial derivative with respect to **each parameter** separately.

Partial derivatives tell us how changing one parameter affects the cost while keeping the others fixed.

Gradient

Definition

The **gradient** is a vector that contains all the partial derivatives of a multivariable function.

For a function of two variables,

$$\nabla f(x, y) = [\partial f / \partial x, \partial f / \partial y]$$

For a function of three variables,

$$\nabla f(x, y, z) = [\partial f / \partial x, \partial f / \partial y, \partial f / \partial z]$$

The symbol ∇ (nabla) represents the gradient operator.

Example

Suppose,

$$f(x, y) = x^2 + y^2$$

Partial derivatives:

$$\partial f / \partial x = 2x$$

$$\partial f / \partial y = 2y$$

Therefore,

$$\nabla f = [2x, 2y]$$

Meaning of the Gradient

The gradient points in the direction of the **maximum increase** of a function.

- Moving **along the gradient** increases the function value.
- Moving **opposite to the gradient (negative gradient)** decreases the function value.

Machine learning uses the **negative gradient** because the goal is to **minimize the cost function**.

Significance of Gradient in Machine Learning

The gradient is used in almost every machine learning algorithm.

Applications include:

- Linear Regression
- Logistic Regression
- Neural Networks
- Deep Learning
- Support Vector Machines
- Gradient Boosting
- Reinforcement Learning

Without gradients, models cannot learn from data.

Gradient Descent

Definition

Gradient Descent is an optimization algorithm used to minimize the cost (loss) function by updating the model parameters in the direction of the **negative gradient**.

Basic Idea

Imagine standing at the top of a hill.

The quickest way to reach the bottom is to move downhill.

Similarly, Gradient Descent repeatedly moves the parameters toward the minimum value of the cost function.

Gradient Descent Formula

Parameter Update Rule:

$$\theta(\text{new}) = \theta(\text{old}) - \alpha \times (\partial J / \partial \theta)$$

where:

θ = Model parameter (weight)

α = Learning Rate

J = Cost Function

$\partial J / \partial \theta$ = Gradient of the cost function

Example

Suppose,

Parameter (θ) = 5

Gradient = 8

Learning Rate = 0.1

New Parameter

$$= 5 - (0.1 \times 8)$$

$$= 5 - 0.8$$

$$= 4.2$$

The parameter moves closer to the value that minimizes the cost.

Learning Rate

Definition

The **Learning Rate** (α) controls how large each update step is during Gradient Descent.

Small Learning Rate

- Training is very slow.
- Requires many iterations.

Large Learning Rate

- May overshoot the minimum.
- Training may become unstable or fail to converge.

Optimal Learning Rate

- Converges quickly.
 - Reaches the minimum efficiently.
-

Types of Gradient Descent

1. Batch Gradient Descent

Uses the **entire training dataset** to calculate the gradient.

Advantages

- Stable updates
- Accurate gradient

Disadvantages

- Slow for large datasets
 - High memory usage
-

2. Stochastic Gradient Descent (SGD)

Uses **one training example** at a time.

Advantages

- Faster updates
- Low memory usage

Disadvantages

- Noisy updates
 - May fluctuate around the minimum
-

3. Mini-Batch Gradient Descent

Uses a **small batch of training examples** (e.g., 32, 64, or 128 samples).

Advantages

- Faster than Batch Gradient Descent
 - More stable than SGD
 - Most commonly used in Deep Learning
-

Cost Function Optimization

What is a Cost Function?

A **Cost Function** measures how well a machine learning model performs.

- Small Cost → Better predictions
- Large Cost → Poor predictions

The goal of training is to **minimize the cost function**.

Example: Mean Squared Error (MSE)

The cost function used in Linear Regression is

$$J(\theta) = (1 / 2m) \times \sum (\hat{y} - y)^2$$

where,

$J(\theta)$ = Cost Function

m = Number of training examples

$\hat{y}$ = Predicted value

y = Actual value

Σ = Sum of all squared errors

The objective is to minimize $J(\theta)$.

Cost Function Optimization Process

Step 1: Initialize the model parameters randomly.

↓

Step 2: Make predictions using the current parameters.

↓

Step 3: Calculate the Cost Function.

↓

Step 4: Compute the Partial Derivatives.

↓

Step 5: Calculate the Gradient.

↓

Step 6: Update the Parameters using Gradient Descent.

↓

Step 7: Repeat until the Cost Function reaches its minimum value.

Applications in Machine Learning

Gradient-based optimization is used in:

- Linear Regression
 - Logistic Regression
 - Neural Networks
 - Deep Learning
 - Convolutional Neural Networks (CNNs)
 - Recurrent Neural Networks (RNNs)
 - Transformers and Large Language Models (LLMs)
 - Support Vector Machines (SVMs)
 - Recommendation Systems
 - Reinforcement Learning
-

Key Formulas

Partial Derivative

$$\partial f / \partial x$$

Gradient

$$\nabla f = [\partial f / \partial x, \partial f / \partial y, \dots, \partial f / \partial w]$$

Gradient Descent Update Rule

$$\theta(\text{new}) = \theta(\text{old}) - \alpha \times (\partial J / \partial \theta)$$

Mean Squared Error (MSE)

$$J(\theta) = (1 / 2m) \times \sum (\hat{y} - y)^2$$

General Parameter Update

$$\text{New Parameter} = \text{Old Parameter} - \text{Learning Rate} \times \text{Gradient}$$

Summary

Topic	Description
Partial Derivative	Derivative with respect to one variable while keeping others constant
Gradient	Vector containing all partial derivatives

Negative Gradient	Direction of the fastest decrease in the function
Gradient Descent	Algorithm used to minimize the cost function
Learning Rate	Controls the size of each update step
Batch Gradient Descent	Uses the entire dataset
Stochastic Gradient Descent	Uses one sample at a time
Mini-Batch Gradient Descent	Uses small batches of samples
Cost Function	Measures the prediction error of a model

Important Points for Exams

- Partial derivatives are used when a function has multiple variables.
- The gradient is a vector of partial derivatives.
- The gradient points in the direction of maximum increase.
- Machine learning minimizes the cost by moving in the **negative gradient** direction.
- Gradient Descent is the most widely used optimization algorithm in machine learning.
- The learning rate determines how quickly a model learns.
- Mini-Batch Gradient Descent is the most commonly used optimization method in deep learning.
- Cost functions measure prediction error, and training aims to minimize this error.

Previous Pendings

Random sampling techniques (simple, stratified, systematic, cluster, convenience)

Types of matrices (square, rectangular, diagonal, identity, symmetric, skew-symmetric, triangular, sparse)

Matrix properties (distributive, transpose, inverse, determinant properties)

Vector spaces and subspaces Solving systems of linear equations Eigenvalues, eigenvectors, and matrix diagonalization

PCA mathematical intuition and implementation

Limits and continuity

Gradient Descent derivation and implementation

Practice identifying matrix types, their properties, and Gradient Descent